

Reviewer Pack — Minimal Topological Entropy and DPLL Proof Length Correlatio...

Entry c32d771bae7d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 15:06:38 UTC

Minimal Topological Entropy and DPLL Proof Length Correlation

Entry ID: c32d771bae7d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 15:06:38 UTC

1. Conjecture

Field A (mathematical branch): Topological Dynamics **Field B** (complexity object): DPLL Search Trees

Statement:

For every Boolean satisfiability instance φ , the minimal topological entropy ($h(\varphi)$) of its associated search tree is linearly correlated with its DPLL proof length $L(\varphi)$, such that $h(\varphi) = \Theta(L(\varphi))$.

Rationale (proposer's reasoning):

Topological dynamics can reveal underlying structure in complex systems, and if a connection between the entropy of a dynamical system and the complexity of solving an associated problem holds, it may suggest new insights into computational hardness. The topological entropy captures the complexity of the dynamical behavior of the search tree, which could be related to the DPLL proof length.

Taxonomy category: topological_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e16a568c5bfe9a7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Boolean satisfiability instance φ with n variables ($n \leq 40$), if the Pearson correlation coefficient $r(h(\varphi), L(\varphi))$ is greater than or equal to 0.7 and there are no instances where $|h(\varphi) - L(\varphi)| > 10$ across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -topological dynamics AND DPLL proof length-minimal topological entropy IN BOOLEAN satisfiability instances AND search tree structure- correlation between $h(\phi)$ and $L(\phi)$ IN Boolean satisfiability problems AND DPLL algorithm

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_random_formula(n: int) -> list:
    literals = [f'x{i}' for i in range(1, n+1)]
    clauses = []
    for _ in range(n):
        clause = random.sample(literals + [f'~{lit}' for lit in literals], 2)
        clauses.append(clause)
    return clauses

def dpll_search_tree(formula: list) -> int:
    def solve(lits_true, lits_false):
        if not formula:
            return 1
        literal = random.choice([l for l in lits_true + lits_false if l[0] != '~'])
        new_lits_true = [lit for lit in lits_true if lit != literal and f'~{lit}' not in lits_false]
        new_lits_false = [lit for lit in lits_false if lit != literal and f'~{lit}' not in lits_true]
        return solve(new_lits_true, new_lits_false) + solve(new_lits_false, new_lits_true)
    return solve([f'x{i}' for i in range(1, len(formula)+1)], [f'~x{i}' for i in range(1, len(formula)+1)])

def topological_entropy(n: int) -> float:
    if n == 0:
        return 0
    return math.log2(n)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    instances_tested = 0
    n_max = 0
```

```

for n in [5, 10, 15, 20, 30, 40]:
    formula = generate_random_formula(n)
    h_phi = topological_entropy(len(formula))
    L_phi = dpll_search_tree(formula)

    if h_phi == 0 or L_phi == 0:
        continue

    results.append((h_phi, L_phi))
    instances_tested += 1
    n_max = max(n_max, n)

if not results:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

mean_x = sum(x for x, _ in results) / len(results)
mean_y = sum(y for _, y in results) / len(results)
numerator = sum((x - mean_x) * (y - mean_y) for x, y in results)
denominator = math.sqrt(sum((x - mean_x)**2 for x, _ in results) * sum((y - mean_y)**2 for _, y in results))

if denominator == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "Denominator is zero"
    }

correlation_coefficient = numerator / denominator

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) >= 0.7 and all(abs(h_phi - L_phi) <= 10 for h_phi, L_phi in results),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

    total_results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"n_max\": {n_max}, \"instances_tested\": {instances_tested}, \"conjecture_holds\": {conjecture_holds}, \"counterexample\": \"{counterexample}\"}}")

```


| next: Review the code for potential bugs that could cause an IndexError and ensure the test can run to completion.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14483
2	preregistration	ollama_remote	glm4:latest	0	0	12226
3	novelty	ollama_remote	glm4:latest	0	0	9075
4	novelty	ollama_remote	glm4:latest	0	0	8869
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15285
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10750
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10758
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14084
9	judge	ollama_remote	glm4:latest	0	0	14518

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110047 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c32d771bae7d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c32d771bae7d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c32d771bae7d.tar.gz` (if generated)